Boolean Algebra & Logic Circuits, Part III

Symmetric Boolean Functions

Chee Wei Tan

Two Functions from Parts I–II

$$\text{full-adder sum} = x \oplus y \oplus z, \qquad \text{carry} = \mathrm{maj}(x, y, z).$$

Common property

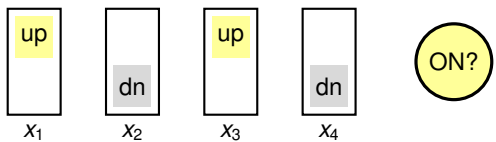
Neither depends on *which* inputs are 1, only on *how many*.

Functions with this property are called **symmetric**.

Outline

1. Counting switches: definition, value vector, Shannon's lattice and the BDD, the full adder as a counter
2. Threshold gates: one gate, two layers, a lower bound, Minnick's trick
3. An algebra of k-numbers: union, intersection, splitting, the parity bit of a sum
4. The XOR basis: Lucas' theorem, periodicity, Walsh spectra, bent functions
5. Beyond parity: exact weight identities and mod-2^t checks

Four Light Switches



- "On iff switch 1 is up": depends on *which*.
- "On iff exactly two are up": depends only on *how many*. ← symmetric

Definition

Symmetric Boolean function

$f : \{0,1\}^n \to \{0,1\}$ is **symmetric** if for every permutation π of $\{1,\ldots,n\}$

$$f(x_1,\ldots,x_n) = f(x_{\pi(1)},\ldots,x_{\pi(n)}).$$

Equivalent form

f is symmetric iff $f(x)$ depends only on the **weight** $\mathrm{wt}(x) = x_1 + \cdots + x_n$.

Familiar Examples

function	on iff . . .
AND	all n inputs are 1
OR	at least one is 1
parity (XOR)	an odd number are 1
majority	more than half are 1
“exactly k”	exactly k are 1

The Value Vector

One bit per weight:

$$\mathbf{v}(f) = (v_0, v_1, \ldots, v_n), \qquad v_k = f(x) \text{ for any } x \text{ with } \mathrm{wt}(x) = k.$$

$n = 4$	v_0	v_1	v_2	v_3	v_4
AND	0	0	0	0	1
OR	0	1	1	1	1
parity	0	1	0	1	0
“at least 2”	0	0	1	1	1
“exactly 2”	0	0	1	0	0

How Many?

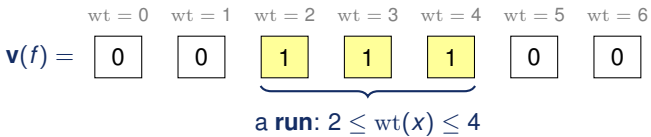
wt	f
0	*
1	*
2	*
3	*
4	*

$$\#\{\text{symmetric functions of } n \text{ variables}\} = 2^{n+1}$$

against 2^{2^n} Boolean functions in all.

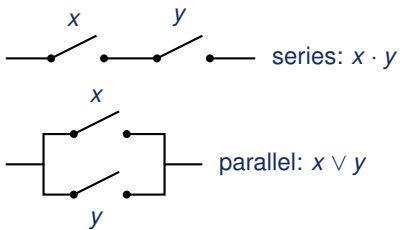
$n = 4$: 32 symmetric among 65 536.

The Weight Line



- Every symmetric function is a union of runs of ones.
- Parity: every other cell, $\lceil n/2 \rceil$ runs of length one.

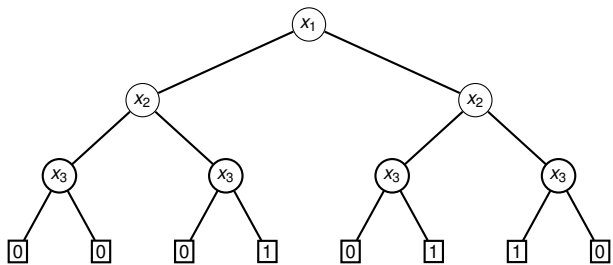
Shannon, 1938: Switches in Series and Parallel



- A **make** contact x closes when $x = 1$; a **break** contact $\bar{x}$ closes when $x = 0$.
- Series $=$ AND, parallel $=$ OR.
- Cost $=$ number of contacts.

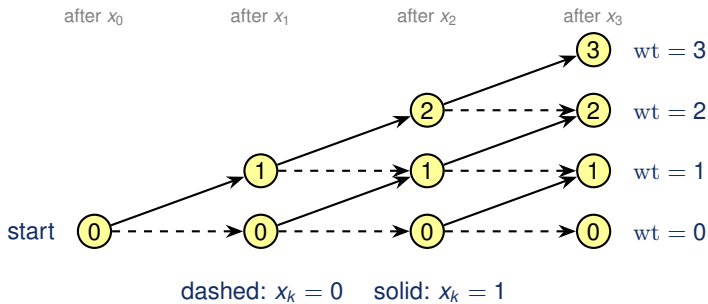
Shannon's thesis asked: how many contacts does a given function need? Symmetric functions were his first worked class.

A Decision Tree for “Exactly Two of Three”



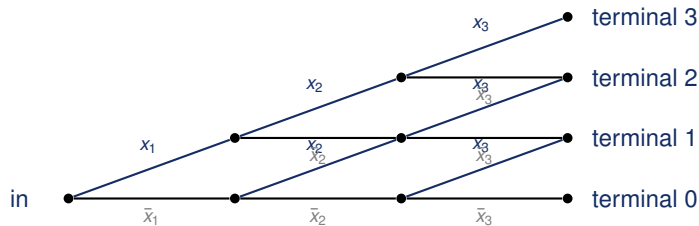
Left branch $= 0$, right branch $= 1$. $2^n - 1$ decision nodes, 2^n leaves. The two middle x_3 subtrees are identical: both have seen *one* 1.

Merge Nodes That Have Seen the Same Count



Node (k, j): k variables read, j of them 1. Only j matters for a symmetric function, so the tree folds into a triangle with $\binom{n+2}{2}$ nodes instead of $2^{n+1} - 1$.

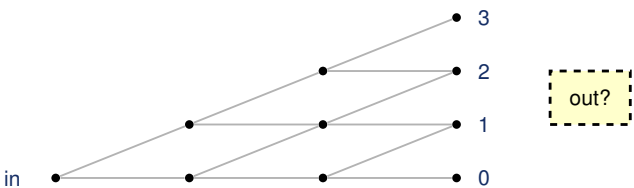
Shannon's Lattice for Symmetric Functions



Replace each edge by a contact: x_k on the rising edge, $\bar{x}_k$ on the level edge. Current reaches terminal j iff $\mathrm{wt}(x) = j$.

S_K: wire the terminals in K to the output. Contacts: $2(1 + 2 + \cdots + n) = n(n+1)$, against $2(2^n - 1)$ for the tree.

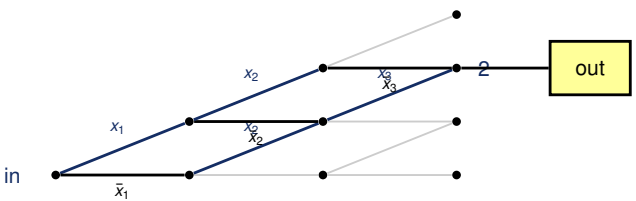
Your Turn 0: “Exactly Two of Three” on the Lattice



Task

Which terminal(s) connect to the output? Which contacts can then be deleted because no current through them can reach the output? How many contacts remain?

Answer 0



Connect terminal 2 only. Keep an edge iff it lies on a path from “in” to terminal 2: **7 contacts**, against $n(n+1) = 12$ in the full lattice and 14 in the tree.

Grey edges carry no current that can reach the output and are deleted.

The Lattice Is the BDD of Part I

- Read $x_1, x_2, \ldots, x_n$ in order; node (k, j) holds the subfunction "f with j ones already seen". Dashed edge $=$ low branch, solid $=$ high branch.
- Merging equal subfunctions is exactly the BDD reduction rule. For a symmetric f the subfunction at level k depends only on j: at most $k + 1$ nodes per level, $O(n^2)$ nodes in all.
- Parity: nodes (k, j) and (k, j') are equal whenever $j \equiv j' \pmod 2$; two nodes per level. That is the XOR BDD of Part I.

Observation

If the value vector repeats every m cells, only the count *mod* m matters, so the BDD needs at most m nodes per level. Parity is the case $m = 2$.

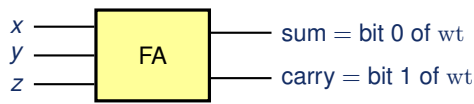
Three Pictures of One Function

value vector	weight line	lattice / BDD
$(v_0, \ldots, v_n)$	runs of ones	paths from “in” to the terminals in K
$n+1$ bits	$T(f)$ transitions	$\leq n(n+1)$ contacts, $O(n^2)$ nodes

Section 2 counts gates, Section 3 does algebra on K, Section 4 reads degree off the periodicity of the same $n+1$ bits.

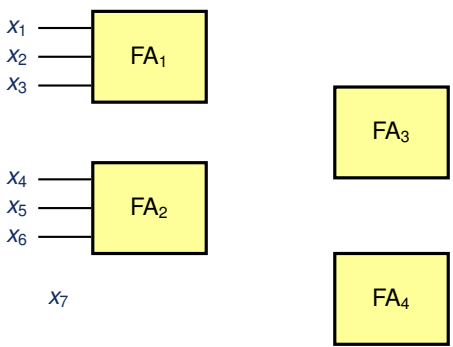
The Full Adder Is a 3-Input Counter

$\mathrm{wt}(x, y, z)$	sum	carry
0	0	0
1	1	0
2	0	1
3	1	1



(carry, sum) is the two-bit binary number $x + y + z$.

Your Turn 1: Count the Ones Among Seven Inputs

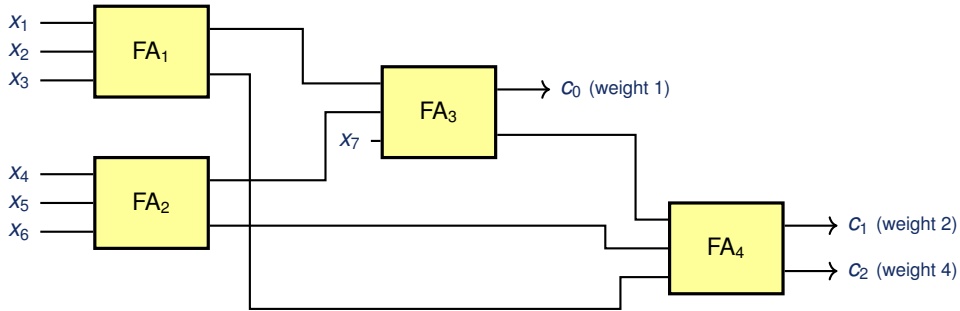


Task

Wire the four full adders so that $c_2 c_1 c_0 = x_1 + \cdots + x_7$ in binary.

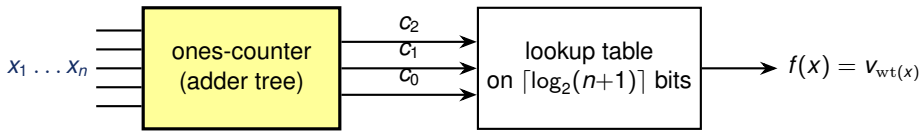
Hint: a full adder turns three bits of weight 2^i into one bit of weight 2^i and one of weight 2^{i+1}.

Answer 1: The 7-to-3 Counter



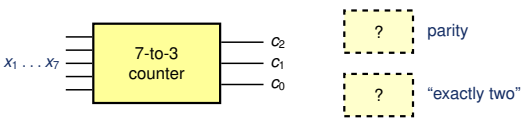
Each full adder removes one bit: n inputs need about n full adders.

Any Symmetric Function = Counter + Lookup



Every symmetric function has an AND/OR/NOT circuit of size $O(n)$.

Your Turn 2: Two Functions from One Counter



Task

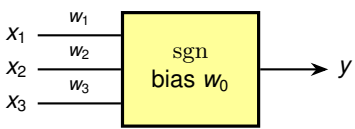
From $c_2 c_1 c_0$ build (1) the parity of the seven inputs and (2) “exactly two”. How many gates each?

Answer 2

- Parity $= c_0$: zero gates.
- Exactly two $= \bar{c}_2\, c_1\, \bar{c}_0$: one AND, two inverters.

After counting, every symmetric function is a function of $\lceil \log_2(n+1) \rceil$ bits.

The Linear Threshold (LT) Gate



$$y = \mathrm{sgn}\Big(w_0 + \sum_{i=1}^{n} w_i x_i\Big)$$

$\mathrm{sgn}(t) = 1$ if $t \geq 0$, else 0.

- AND: weights 1, bias $-n$. OR: weights 1, bias -1. $x_1 \bar{x}_2$: weights $1, -1$, bias -1.

Depth-2 Size of XOR

	$\mathrm{XOR}(x_1, x_2, x_3)$	$\mathrm{XOR}(x_1, \ldots, x_n)$
AND/OR/NOT	5	$2^{n-1} + 1$
LT, layered	4	$n + 1$
LT, non-layered	2	$\lfloor n/2 \rfloor + 1$

Layered: inputs feed only the first layer. **Non-layered**: inputs may reach any layer.

All right-column entries are optimal.

Threshold Functions: the Class TH

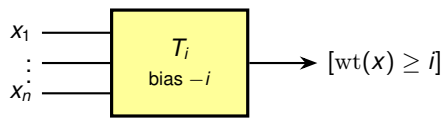
$$T_i(x) = 1 \iff \mathrm{wt}(x) \geq i, \qquad \bar{T}_i(x) = 1 \iff \mathrm{wt}(x) < i.$$

wt	T_0	T_1	T_2	T_3	$\bar{T}_0$	$\bar{T}_1$	$\bar{T}_2$	$\bar{T}_3$
0	1	0	0	0	0	1	1	1
1	1	1	0	0	0	0	1	1
2	1	1	1	0	0	0	0	1
3	1	1	1	1	0	0	0	0

TH: symmetric functions with at most one transition. $|\mathbf{TH}| = 2n + 2$.

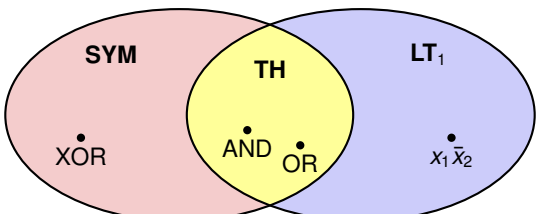
Every TH Function Is One Gate

$$T_i = \mathrm{sgn}\big(-i + x_1 + \cdots + x_n\big), \qquad \bar{T}_i = \mathrm{sgn}\big((i-1) - x_1 - \cdots - x_n\big)$$



Question: with arbitrary weights, can one LT gate compute more symmetric functions than TH?

Theorem: SYM ∩ LT_1 = TH



A symmetric function is one LT gate iff it has at most one transition.

Proof: Shrink to Two Variables

Suppose g is symmetric, one LT gate, with two or more transitions. Its table contains $0, 1, \ldots, 1, 0$ (or the complement).

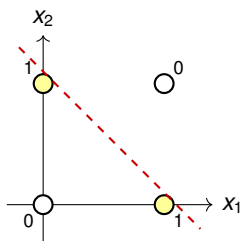
$g = 0\,0\,1\,0\,1\,1$ on $n = 5$

Fix $x_3 = 1$, $x_4 = x_5 = 0$: $\mathrm{wt}(x) = x_1 + x_2 + 1 \in \{1, 2, 3\}$ and g reads $0, 1, 0$.

$x_1 x_2$	$\mathrm{wt}(x)$	g
00	1	0
01, 10	2	1
11	3	0

$$\mathrm{sgn}\big((w_0 + w_3) + w_1 x_1 + w_2 x_2\big) = \mathrm{XOR}(x_1, x_2)$$

Proof: XOR Is Not a Line



One LT gate in two variables draws one line $w_1 x_1 + w_2 x_2 = \text{const}$ and outputs 1 on one side.

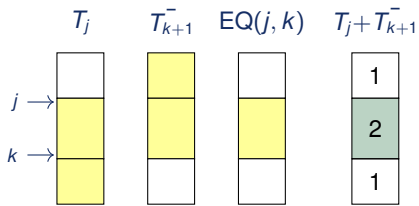
No line puts $\{01, 10\}$ on one side and $\{00, 11\}$ on the other.

Contradiction. □

Two Gates Make an Interval

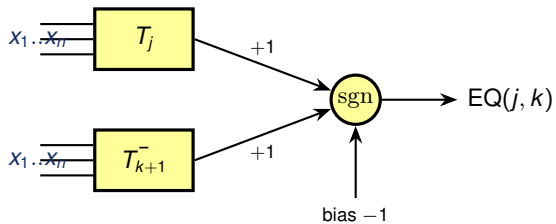
$$\mathrm{EQ}(j,k)(x) = 1 \iff j \le \mathrm{wt}(x) \le k$$

$$\boxed{\mathrm{EQ}(j,k) = T_j + \bar{T}_{k+1} - 1}$$



shaded $= 1$, white $= 0$; rows: weights below j, from j to k, above k

The Interval Circuit



Depth 2, three gates, for any single run. $\mathrm{XOR}_2 = \mathrm{EQ}(1,1) = T_1 + \bar{T}_2 - 1$.

The Layered Design Algorithm

$$f = \sum_{\text{runs } (j,k)} \left(T_j + \bar{T}_{k+1} - 1 \right)$$

Rule

- $0 \Rightarrow 1$ transition at weight i: gate T_i
- $1 \Rightarrow 0$ transition at weight i: gate $\bar{T}_i$
- -1 for every run with a 0 on both sides

$T(f)$ first-layer gates ($T(f)$ = number of transitions), one output gate: $T(f) + 1$.

Worked Example, $n = 7$

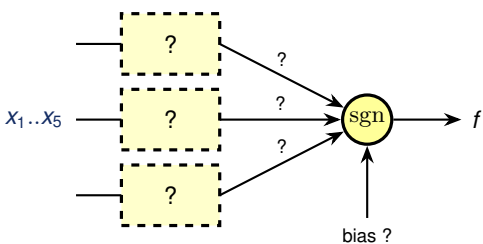
wt	0	1	2	3	4	5	6	7
f	1	1	0	1	1	0	1	1
gate			$\bar{T}_2$	T_3		$\bar{T}_5$	T_6	

$$f = \bar{T}_2 + T_3 + \bar{T}_5 + T_6 - 1$$

Check $\mathrm{wt} = 4$: $0 + 1 + 1 + 0 - 1 = 1$ ✓ $\mathrm{wt} = 7$: $0 + 1 + 0 + 1 - 1 = 1$ ✓ $T(f) = 4$, five gates.

Your Turn 3: Fill in the Gates

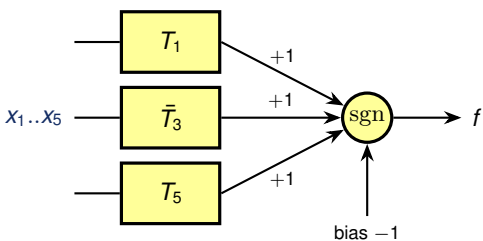
wt	0	1	2	3	4	5
f	0	1	1	0	0	1



Task

Value vector 0 1 1 0 0 1 on five inputs. Which TH gates, which weights, which bias?

Answer 3



$$f = T_1 + \bar{T}_3 + T_5 - 1$$

Transitions at weights 1, 3, 5: $T(f) = 3$. Run $[1, 2]$ is isolated (-1); run $[5, 5]$ touches the end (no -1).

$\mathrm{wt} = 2$: $1 + 1 + 0 - 1 = 1$ ✓ $\mathrm{wt} = 4$: $1 + 0 + 0 - 1 = 0$ ✓

XOR Needs n Gates in the First Layer

$$\mathrm{XOR}_5 = (T_1 + \bar{T}_2 - 1) + (T_3 + \bar{T}_4 - 1) + T_5$$

wt	0	1	2	3	4	5
XOR	0	1	0	1	0	1
gate		T_1	$\bar{T}_2$	T_3	$\bar{T}_4$	T_5

$T(f) = n$: $n + 1$ gates.

Lower Bound

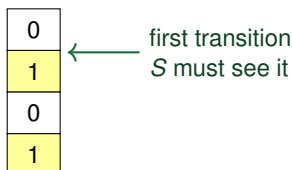
Theorem

A layered LT circuit for a symmetric f with a TH first layer has at least $T(f) + 1$ gates ($T(f) > 1$), at any depth.

Base $T(f) = 2$: $f \notin$ **TH**, so two gates at least; a gate fed by one wire is identity or complement, hence redundant; so at least three.

Lower Bound: the Sensitive Gate

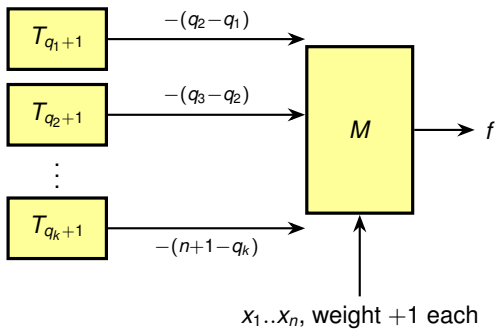
- First transition of f: weight $i \to i+1$.
- Some first-layer gate S changes between $\mathrm{wt} = i$ and $i+1$; otherwise the circuit cannot tell $f(i) \neq f(i+1)$.
- Set $x_1 = \cdots = x_{i+1} = 1$: S is constant, delete it. Remaining: $k-1$ gates for a symmetric g with $T(g) = T(f) - 1$.
- Induction: $k - 1 \geq T(g) + 1$, so $k \geq T(f) + 1$. $\square$



Minnick's Idea: Feed the Count to the Output Gate

Ones of the table at weights $q_1 < \cdots < q_k$. One threshold T_{q_i+1} per one, plus

$$M = \operatorname{sgn}\Big(\operatorname{wt}(x) - q_1 - \sum_{i<k}(q_{i+1} - q_i)\, T_{q_i+1} - (n{+}1{-}q_k)\, T_{q_k+1}\Big)$$



Why Minnick's Gate Works

- At $\mathrm{wt} = q_j$: subtracted thresholds total $q_j - q_1$; argument $= 0$; output 1.
- For $q_j < \mathrm{wt} < q_{j+1}$: argument $= \mathrm{wt} - q_{j+1} < 0$; output 0.
- Below q_1: argument $= \mathrm{wt} - q_1 < 0$. Above q_k: argument $= \mathrm{wt} - n - 1 < 0$.

Gates: k thresholds $+ M$; one fewer if $q_k = n$.

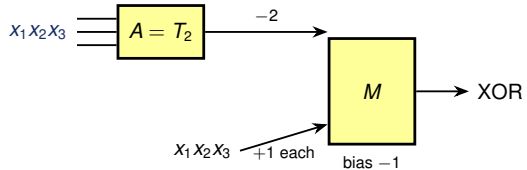
XOR of Three in Two Gates

$q = (1, 3)$, $n = 3$; the threshold for $q_2 = n$ vanishes.

$$A = \operatorname{sgn}(x_1 + x_2 + x_3 - 2)$$

$$\mathrm{XOR} = \operatorname{sgn}(-1 - 2A + x_1 + x_2 + x_3)$$

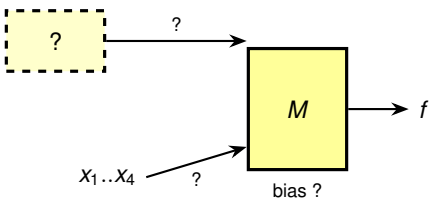
wt	A	$-1 - 2A + \mathrm{wt}$	XOR
0	0	-1	0
1	0	0	1
2	1	-1	0
3	1	0	1



XOR_n: $\lceil n/2 \rceil$ ones, $\lfloor n/2 \rfloor + 1$ gates.

Your Turn 4: Minnick for “Two or Four of Four”

wt	0	1	2	3	4
f	0	0	1	0	1



Task

$n = 4$, ones at $q = (2, 4)$. Which threshold, which weights, which bias? How many gates does the layered algorithm need?

Answer 4

$q_2 = 4 = n$: last threshold vanishes. T_3 with weight -2, bias -2:

$$f = \operatorname{sgn}\big(x_1 + x_2 + x_3 + x_4 - 2 - 2\,T_3(x)\big)$$

wt	T_3	$\mathrm{wt} - 2 - 2T_3$	f
0	0	-2	0
1	0	-1	0
2	0	0	1
3	1	-1	0
4	1	0	1

Gate count

Minnick: 2 gates.

Layered: T_2, $\bar{T}_3$, T_4 + output $= 4$ gates.

The S-Notation

Definition

For a set of k-**numbers** $K \subseteq \{0, \ldots, n\}$,

$$S_K(x) = S_{k_1 \vee k_2 \vee \cdots \vee k_m}(x) = 1 \iff \mathrm{wt}(x) \in K.$$

S_k alone: "exactly k". $K = \{k : v_k = 1\}$, the support of the value vector.

k-Numbers of Familiar Functions

function ($n = 3$)	value vector	S-notation
full-adder sum	0 1 0 1	$S_{1\vee 3}$
carry (majority)	0 0 1 1	$S_{2\vee 3}$
AND	0 0 0 1	S_3
OR	0 1 1 1	$S_{1\vee 2\vee 3}$
“at most one”	1 1 0 0	$S_{0\vee 1}$

Parity of n bits: $S_{1\vee 3\vee 5\vee\cdots}$.

OR, AND, NOT Are Set Operations

$$S_K \vee S_L = S_{K \cup L}, \qquad S_K \cdot S_L = S_{K \cap L}, \qquad \overline{S_K} = S_{\{0,\ldots,n\} \setminus K}.$$

$n = 3$

- $S_{1 \vee 3} \cdot S_{2 \vee 3} = S_3$
- $S_{1 \vee 3} \vee S_{2 \vee 3} = S_{1 \vee 2 \vee 3}$
- $\overline{S_{2 \vee 3}} = S_{0 \vee 1}$

The symmetric functions form a Boolean algebra isomorphic to the subsets of $\{0, \ldots, n\}$.

Shannon Expansion for S

$$\boxed{S_K(x_1, \ldots, x_n) = x_1 \cdot S_{K-1}(x_2, \ldots, x_n) \ \vee \ \bar{x}_1 \cdot S_K(x_2, \ldots, x_n)}$$

$$K - 1 = \{k - 1 : k \in K, \ k \geq 1\}$$

Setting $x_1 = 1$ uses one unit of weight: the k-numbers shift down by one.

Example: Parity of Three

$K = \{1, 3\}$, $K - 1 = \{0, 2\}$:

$$S_{1 \vee 3}(x_1, x_2, x_3) = x_1 \, S_{0 \vee 2}(x_2, x_3) \;\vee\; \bar{x}_1 \, S_{1 \vee 3}(x_2, x_3)$$

On two variables $S_{0 \vee 2} = \overline{x_2 \oplus x_3}$ and $S_{1 \vee 3} = S_1 = x_2 \oplus x_3$:

$$= x_1 \, \overline{(x_2 \oplus x_3)} \;\vee\; \bar{x}_1 \, (x_2 \oplus x_3) = x_1 \oplus x_2 \oplus x_3. \; \checkmark$$

A k-number larger than the number of remaining variables drops out.

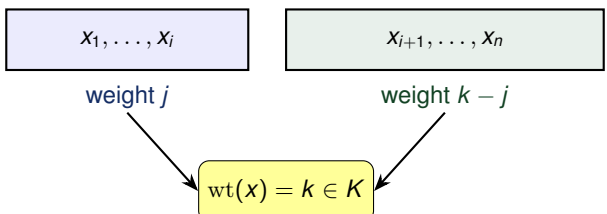
The Splitting Theorem

Blocks $x_I = (x_1, \ldots, x_i)$ and $x_J = (x_{i+1}, \ldots, x_n)$:

$$S_K(x) = \bigvee_{j=0}^{i} S_j(x_I) \cdot S_{K-j}(x_J), \qquad K - j = \{k - j : k \in K,\ k \geq j\}.$$

Proof: induction on i with the expansion of the previous slide.

Splitting, Drawn



A Boolean Vandermonde convolution: $\binom{n}{k} = \sum_j \binom{i}{j}\binom{n-i}{k-j}$ counts the same split.

Example: Majority Split as $1 + 2$

$$S_{2\vee 3}(x_1; x_2, x_3) = S_0(x_1)\, S_{2\vee 3}(x_2, x_3) \;\vee\; S_1(x_1)\, S_{1\vee 2}(x_2, x_3)$$
$$= \bar{x}_1 \cdot x_2 x_3 \;\vee\; x_1 \cdot (x_2 \vee x_3) \;\checkmark$$

Odd k-Numbers: Parity of Parities

$K = \{1, 3, 5, \dots\}$. Split and sort by whether j is odd or even:

$$S_{\text{odd}}(x) = S_{\text{even}}(x_I) \cdot S_{\text{odd}}(x_J) \;\vee\; S_{\text{odd}}(x_I) \cdot S_{\text{even}}(x_J).$$

With $S_{\text{even}} = \overline{S_{\text{odd}}}$:

$$S_{\text{odd}}(x_1, \dots, x_n) = S_1\big[\, S_{\text{odd}}(x_I),\; S_{\text{odd}}(x_J) \,\big]$$

Parity is the XOR of the block parities.

Parity Composes; Majority Does Not

Any number of blocks

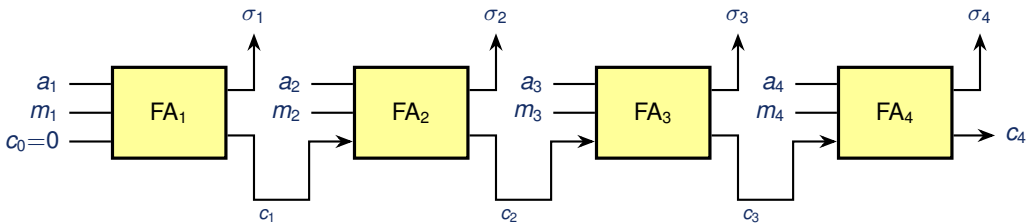
$$S_{\mathrm{odd}}(x) = S_{\mathrm{odd}}(p_1, p_2, \ldots, p_q),$$

p_t the parity of block t.

Majority

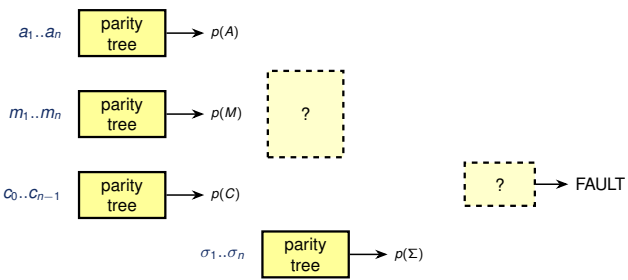
Blocks 110, 110, 000: block majorities $1, 1, 0$, majority of those $= 1$; but only 4 of 9 inputs are 1.

A Ripple-Carry Adder



$$\sigma_i = S_{1\vee 3}(a_i, m_i, c_{i-1}), \qquad c_i = S_{2\vee 3}(a_i, m_i, c_{i-1}).$$

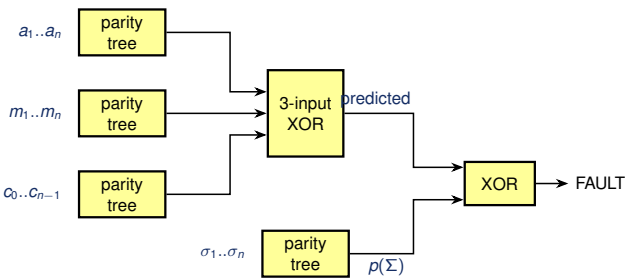
Your Turn 5: Predict the Parity of the Sum



Task

Fill the first box so that it predicts $p(\Sigma)$ from $p(A), p(M), p(C)$; fill the second so that FAULT $= 1$ iff the adder's parity disagrees.

Answer 5



Parity theorem

$$p(\Sigma) = p(A) \oplus p(M) \oplus p(C)$$

Predictor: 3-input XOR.
Comparator: XOR.

Why the Parity Theorem Holds

Each $\sigma_i = S_{1\vee 3}(a_i, m_i, c_{i-1})$ is a parity, so

$$p(\Sigma) = S_{\text{odd}}(\sigma_1, \ldots, \sigma_n) = S_{\text{odd}}\big(a_1, m_1, c_0,\ \ldots,\ a_n, m_n, c_{n-1}\big).$$

Regroup the $3n$ bits into the a's, the m's and the c's:

$$p(\Sigma) = S_{1\vee 3}\big[\, p(A),\ p(M),\ p(C)\,\big] = p(A) \oplus p(M) \oplus p(C). \quad \square$$

Radix r: $\sigma_i \equiv a_i + m_i + c_{i-1} \pmod r$, so $\sum \sigma_i \equiv \sum a_i + \sum m_i + \sum c_{i-1} \pmod r$.

Example: $11 + 6 = 17$

i	1	2	3	4	parity
a_i	1	1	0	1	$p(A) = 1$
m_i	0	1	1	0	$p(M) = 0$
c_{i-1}	0	0	1	1	$p(C) = 0$
σ_i	1	0	0	0	$p(\Sigma) = 1$

$c_4 = 1$

$1 \oplus 0 \oplus 0 = 1 = p(\Sigma)$ ✓ Flip σ_2 (stuck bit): $p(\Sigma) = 0$, prediction 1, FAULT $= 1$.

Limits of the k-Number Algebra

- Disjunctive basis: $\vee$ of "exactly k".
- Gives: set operations, splitting, parity composition.
- Does not give: algebraic degree, distance to affine functions, correlation with linear functions.

For those: the XOR basis.

Elementary Symmetric Polynomials

$$\sigma_i(x) = \bigoplus_{1 \le j_1 < \cdots < j_i \le n} x_{j_1} x_{j_2} \cdots x_{j_i}$$

$\sigma_0 = 1$, $\sigma_1 = \text{parity}$, $\sigma_2 = \bigoplus_{j<l} x_j x_l$, $\sigma_n = \text{AND}$.

At weight k: $\sigma_i(x) = \binom{k}{i} \bmod 2$.

Two Bases

Simplified ANF

f symmetric iff $f = \bigoplus_{i=0}^{n} \lambda_i \sigma_i$ for some $\boldsymbol{\lambda}(f) = (\lambda_0, \ldots, \lambda_n) \in \mathbb{F}_2^{n+1}$.

- $\mathbf{v}(f)$: values by weight (support $=$ the k-numbers).
- $\boldsymbol{\lambda}(f)$: coefficients by degree; $\deg f = \max\{i : \lambda_i = 1\}$.

Lucas' Theorem Links Them

$\binom{k}{i}$ is odd iff the binary digits of i are a subset of those of k; write $i \preceq k$.

$$v_k = \bigoplus_{i \preceq k} \lambda_i, \qquad \lambda_i = \bigoplus_{k \preceq i} v_k.$$

Self-inverse (Möbius inversion over the subset order); $O(n \log n)$ XORs.

Lucas in Action, $n = 4$

	$\mathbf{v}$	$\boldsymbol{\lambda}$
parity	0 1 0 1 0	0 1 0 0 0 $(= \sigma_1)$
T_2	0 0 1 1 1	0 0 1 0 1 $(= \sigma_2 \oplus \sigma_4)$
exactly 2	0 0 1 0 0	0 0 1 1 0 $(= \sigma_2 \oplus \sigma_3)$
OR	0 1 1 1 1	0 1 1 1 1

Check "exactly 2" at $k = 3 = 11_2$: $i \preceq 3 \Rightarrow i \in \{0, 1, 2, 3\}$, $v_3 = 0 \oplus 0 \oplus 1 \oplus 1 = 0$ ✓

Majority of three: $\mathbf{v} = 0\,0\,1\,1 \Rightarrow \boldsymbol{\lambda} = 0\,0\,1\,0$, i.e. $\mathrm{maj} = \sigma_2$.

Degree and Periodicity

Theorem

$\deg f < 2^t$ iff $\mathbf{v}(f)$ is periodic with period 2^t: $v_{k+2^t} = v_k$.

Why: if $\lambda_i = 0$ for $i \geq 2^t$, only digits below position t matter in $i \preceq k$.

$n = 6$

$\sigma_1 \oplus \sigma_2$ (degree 2): $\mathbf{v} = 0\,1\,1\,0\,0\,1\,1$, period 4. ✓
σ_4 (degree 4): $\mathbf{v} = 0\,0\,0\,0\,1\,1\,1$, period 8.

The Walsh Transform

$$W_f(a) = \sum_{x \in \{0,1\}^n} (-1)^{f(x) \oplus a \cdot x}$$

- $W_f(0) = 0$ iff f balanced.
- Nonlinearity $= 2^{n-1} - \frac{1}{2} \max_a |W_f(a)|$.
- t-resilient: balanced and $W_f(a) = 0$ for $1 \leq \mathrm{wt}(a) \leq t$.

Walsh Spectrum from $n+1$ Numbers

For symmetric f, $W_f(a)$ depends only on $w = \mathrm{wt}(a)$:

$$W_f(a) = \sum_{k=0}^{n} (-1)^{v_k} K_k(w,n), \qquad K_k(w,n) = \sum_j (-1)^j \binom{w}{j}\binom{n-w}{k-j}$$

(Krawtchouk polynomials, as in the MacWilliams identities.)

Majority of 3, $\mathbf{v} = 0\,0\,1\,1$

$W(w) = (0, 4, 0, -4)$. Balanced; nonlinearity $4 - 2 = 2$.

Bent Symmetric Functions

Bent (n even): $|W_f(a)| = 2^{n/2}$ for all a.

Theorem (Savický)

The symmetric bent functions are exactly

$$\sigma_2, \qquad \sigma_2 \oplus 1, \qquad \sigma_2 \oplus \sigma_1, \qquad \sigma_2 \oplus \sigma_1 \oplus 1.$$

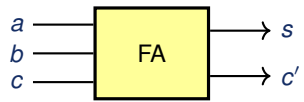
$n = 4$, $f = \sigma_2$, $\mathbf{v} = 0\,0\,1\,1\,0$: $W(w) = (-4, 4, 4, -4, -4)$. ✓

What Symmetry Allows

- Balanced: majority (n odd), parity, and more.
- Resilient: beyond small orders only affine functions.
- Algebraic immunity: majority attains the maximum $\lceil n/2 \rceil$.

$$\begin{array}{ccccc} \text{low degree} & \longleftrightarrow & \text{high nonlinearity} & \longleftrightarrow & \text{high degree} \\ \text{short period} & \longleftrightarrow & \text{bent: } \sigma_2 \text{ only} & \longleftrightarrow & T_k, \text{ majority} \end{array}$$

Counting Ones Through One Full Adder



$$a + b + c = s + 2c' \quad \Longrightarrow \quad \text{ones in} - \text{ones out} = (a + b + c) - (s + c') = c'.$$

In words

A full adder loses exactly one 1 when it carries, and none when it does not. Same for a half adder.

Counting Ones Through the Whole Adder

Add the one-adder facts along a ripple-carry chain. Every carry is produced once and consumed once, except the carry-out c_n, which leaves the chain:

Fact A

$$\mathrm{wt}(\Sigma) = \mathrm{wt}(A) + \mathrm{wt}(M) - (\text{number of internal carries}) - 2c_n.$$

- Taken mod 2 this is the parity theorem of Section 3.
- As an integer it says more: it counts the carries.

Example of Fact A

$11 + 6 = 17$: $\mathrm{wt}(A) + \mathrm{wt}(M) = 3 + 2 = 5$; carries $c_2 = c_3 = c_4 = 1$, c_4 the carry-out.

$$\mathrm{wt}(\Sigma) = 5 - (c_1 + c_2 + c_3) - 2c_4 = 5 - 2 - 2 = 1 \checkmark \qquad (\Sigma = 0001,\ c_4 = 1)$$

Without overflow ($c_n = 0$): $\mathrm{wt}(A) + \mathrm{wt}(M) - \mathrm{wt}(A + M) =$ number of carries.

Kummer's Theorem (1852)

Theorem

The exponent of the prime p in $\binom{n}{m}$ equals the number of carries when m and $n-m$ are added in base p.

Base 2, with $S_2(x) = \mathrm{wt}(x)$ the binary digit sum:

$$\nu_2\binom{n}{m} = \#\text{carries in } m + (n-m) = S_2(m) + S_2(n-m) - S_2(n).$$

The right-hand side is Fact A with $A = m$, $M = n - m$ and no carry-out.

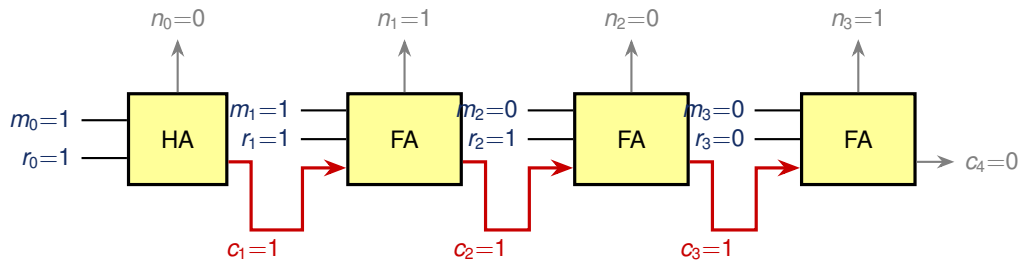
Example: $\binom{10}{3} = 120 = 2^3 \cdot 15$

$m = 3 = 011_2$, $n - m = 7 = 111_2$, $n = 10 = 1010_2$.

bit	3	2	1	0
m	0	0	1	1
$n - m$	0	1	1	1
carry in	1	1	1	0
sum n	1	0	1	0

Three carries $\Rightarrow \nu_2\binom{10}{3} = 3$. Check: $S_2(3) + S_2(7) - S_2(10) = 2 + 3 - 2 = 3$. ✓

The Same Example as a Cascade of Adders



A half adder at bit 0, full adders after it. Kummer counts the red wires and ignores the grey ones.

A Column Is a Full-Adder Truth Table

Column k sees m_k, r_k and the carry-in c_k:

$$n_k = m_k \oplus r_k \oplus c_k, \qquad c_{k+1} = \mathrm{maj}(m_k, r_k, c_k) = m_k r_k + c_k\,(m_k \oplus r_k).$$

- $\nu_2\binom{n}{m}$ is a function of the carry bits $c_1, \ldots, c_\ell$ alone.
- The sum bits n_k never enter: ν_2 is computed by the majority gates, not the XOR gates.

Counting Carries Without Running the Adder

- Ones that go in: $S_2(m) + S_2(r)$. Ones that come out: $S_2(n)$.
- Each adder along the way loses one 1 per carry (previous slides).
- So the number of carries is $S_2(m) + S_2(r) - S_2(n)$, with no ripple simulated.

$\binom{10}{3}$: $S_2(3) + S_2(7) - S_2(10) = 2 + 3 - 2 = 3$ carries, as the cascade showed.

Multinomial Kummer and Carry-Save Adders

$$\nu_2\binom{n}{m_1,\ldots,m_k} = \nu_2\frac{n!}{m_1!\cdots m_k!} = S_2(m_1)+\cdots+S_2(m_k)-S_2(n), \qquad n=\sum m_i.$$

Adding $k \geq 3$ operands in hardware: a **carry-save adder** (CSA) applies full adders column-wise to three operands, producing a sum word and a carry word (shifted left by one); repeat until two words remain, then one ripple addition. Wallace and Dadda trees are CSAs arranged for speed.

The Same Count for Every Adder Design

Fact B

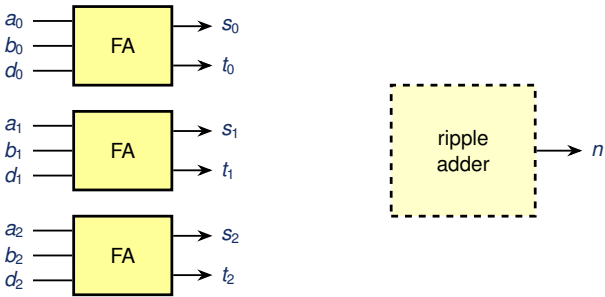
Build $n = m_1 + \cdots + m_k$ from full and half adders in any arrangement. The total number of carries, over all the adders, is always

$$S_2(m_1) + \cdots + S_2(m_k) - S_2(n).$$

Why: ones in $= \sum S_2(m_i)$, ones out $= S_2(n)$, and each adder loses one 1 per carry. The wiring does not matter.

Ripple-carry, carry-save, Wallace, Dadda: same total, differently distributed.

Your Turn 7: Count the Carries in a Carry-Save Adder



Task

$a = b = d = 3 = 011_2$. Fill in the CSA outputs s_i, t_i; wire the ripple adder; count carries in each stage; compare with Fact B.

Answer 7

CSA stage ($a_i = b_i = d_i$ for $i = 0, 1$; zeros at $i = 2$):

i	a_i	b_i	d_i	s_i	t_i
0	1	1	1	1	1
1	1	1	1	1	1
2	0	0	0	0	0

Ripple stage $3 + 6$: $011 + 110$:

i	0	1	2	3
s	1	1	0	0
$t \ll 1$	0	1	1	0
carry in	0	0	1	1
n	1	0	0	1

Sum word $s = 011_2 = 3$, carry word t shifted $= 0110_2 = 6$: 2 carries.

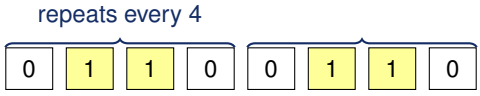
$n = 1001_2 = 9$: 2 carries.

Total $2 + 2 = 4$. Fact B: $S_2(3) + S_2(3) + S_2(3) - S_2(9) = 2 + 2 + 2 - 2 = 4$. ✓

When Is a Block-wise Shortcut Possible?

Section 3: parity of the whole = XOR of block parities. Each block reports one bit, its count mod 2.

Suppose each block may report its count **mod** m. Which symmetric functions can be finished from those reports?



Fact C

Exactly those whose value vector **repeats every** m **cells**. (Blocks must have at least m inputs.)

Why: the whole count mod m is the sum of the block counts mod m; and if the function only needs block counts mod m, adding m ones inside a block cannot change it, so the vector repeats.

Putting Fact C Next to Section 4

Section 4: value vector repeats every 2^t cells $\iff$ algebraic degree $< 2^t$.

- Blocks report counts mod 2 $\leftrightarrow$ degree < 2: only $0, 1, \sigma_1, \sigma_1 \oplus 1$. Parity is the one nontrivial case, which is why Section 3 found nothing else.
- Blocks report counts mod 4 $\leftrightarrow$ degree ≤ 3: σ_2, $\sigma_1 \oplus \sigma_2$, σ_3, ...
- Majority and T_k never repeat: no block-wise shortcut, degree near n.

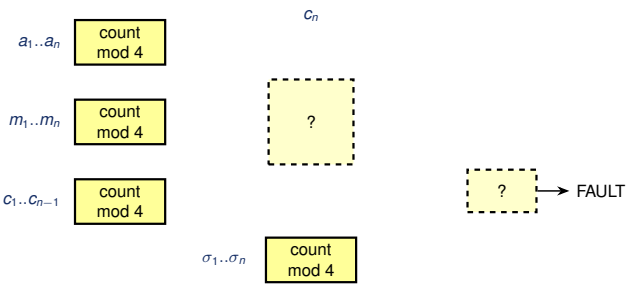
Checking an Adder with Counts Mod 4

Fact A, taken mod 4 instead of mod 2:

$$\mathrm{wt}(\Sigma) \equiv \mathrm{wt}(A) + \mathrm{wt}(M) - (\text{internal carries}) - 2c_n \pmod 4.$$

- Parity check (Section 3): count ones mod 2 in A, M, the carries and Σ; one bit each.
- Mod-4 check: count ones mod 4 instead; two bits each, two-bit adders in place of XORs.
- The same works mod 8, 16, ...

Your Turn 6: The Mod-4 Check



Task

Each counter outputs two bits. What arithmetic goes in the first box? What comparison in the second? Which errors on the sum word does this catch that parity misses?

Answer 6

First box: two-bit arithmetic

$$\widehat{w} = \left(w_A + w_M - w_C - 2c_n\right) \bmod 4.$$

Second box: compare $\widehat{w}$ with $\mathrm{wt}(\Sigma) \bmod 4$; FAULT $= 1$ iff they differ.

What each check catches on the sum word

error on Σ	parity	mod 4
one bit flipped	detects	detects
two bits, same direction	misses	detects
two bits, opposite	misses	misses

Cost: two-bit counters in place of parity trees; the compare is two XORs and an OR.

Summary Table

	threshold gates	***k*-numbers**	**XOR basis**
object	table with $T(f)$ transitions	set $K \subseteq \{0..n\}$	$\mathbf{v}, \boldsymbol{\lambda} \in \mathbb{F}_2^{n+1}$
operations	T_i, $\bar{T}_i$, weights	$\cup, \cap$, complement, splitting	$\oplus$; Lucas between bases
key result	$T(f) + 1$ gates; $\lfloor n/2 \rfloor + 1$ non-layered	$p(\Sigma) = p(A) \oplus p(M) \oplus p(C)$	$\deg < 2^t \iff$ period 2^t

Beyond parity: ones in $-$ ones out $=$ carries (Kummer), for any adder design; block-wise shortcut mod $m \iff$ the value vector repeats every m cells.

Exercises

1. Value vector and k-numbers of (a) “at most one” on $n = 5$, (b) maj_5, (c) $x_1 \oplus \cdots \oplus x_5 \oplus 1$. Gate counts with the layered algorithm and with Minnick’s construction.
2. Wire six full adders into a 15-to-4 counter; then build “exactly 3 of 15” from the four count bits.
3. Expand $S_2(x_1, x_2, x_3, x_4)$ with the splitting theorem, $i = 2$; check at $(1, 1, 0, 0)$ and $(1, 0, 1, 0)$.
4. Prove $S_1(a, S_1(b, c)) = S_{1\vee 3}(a, b, c)$ from the set rules and the Shannon expansion for S.
5. $A = 1101_2$, $M = 1011_2$: carry word, three parities, parity theorem; then Fact A exactly.

Exercises (continued)

6. $\boldsymbol{\lambda}(\mathrm{maj}_5)$ via Lucas; its degree; why its value vector cannot be periodic.
7. $n = 6$: W_f by weight for $f = \sigma_2 \oplus \sigma_1$ via Krawtchouk polynomials; confirm magnitude 8 throughout.
8. Show the only symmetric functions of period 2 are $0, 1, \sigma_1, \sigma_1 \oplus 1$ (Fact C with $m = 2$).
9. Justify the table in Answer 6 and extend it to mod 8. Which triple errors does mod 8 catch?
10. (Harder) Extend Fact B to networks that also contain 4:2 compressors, and to base p (what replaces "one one-bit per carry"?).
11. Draw Shannon's lattice for maj_5 and delete dead contacts. Then show that $\sigma_1 \oplus \sigma_2$ needs exactly 4 BDD nodes per level.

Acknowledgements

- Section 1 follows J. Bruck's Caltech course *IST4 Information and Logic* (lectures 14–18).
- Section 2 follows J. Bruck's Caltech course *CNS 188a Computation Theory and Neural Systems*.
- Section 3 follows N. C. de Troye (1963); Section 4 follows A. Canteaut and M. Videau (2005).

References

- C. E. Shannon, “A symbolic analysis of relay and switching circuits,” *Trans. AIEE*, 57:713–723, 1938 (M.Sc. thesis, MIT, 1937). (Section 1.)
- J. Bruck, *IST4 Information and Logic*, lectures 14–18, Caltech. (Section 1.)
- R. E. Bryant, “Graph-based algorithms for Boolean function manipulation,” *IEEE Trans. Computers*, C-35(8):677–691, 1986.
- S. Muroga, “Logical elements on majority decision principle and complexity of their circuit,” Int. Conf. Information Processing, Paris, 1959.
- R. C. Minnick, “Linear-input logic,” *IRE Trans. Electronic Computers*, EC-10:6–16, 1961.
- N. C. de Troye, “Some properties of symmetric switching functions,” *Philips Research Reports*, 18:331–337, 1963. (Section 3.)
- M. Minsky and S. Papert, *Perceptrons*, MIT Press, 1969.
- K.-Y. Siu, V. Roychowdhury, T. Kailath, *Discrete Neural Computation*, Prentice Hall, 1995.
- J. Bruck, *CNS 188a Computation Theory and Neural Systems*, Lecture 6, Caltech, 2006. (Section 2.)

References (continued)

- A. Canteaut and M. Videau, “Symmetric Boolean functions,” *IEEE Trans. Information Theory*, 51(8):2791–2811, 2005. (Section 4.)
- P. Savický, “On the bent Boolean functions that are symmetric,” *European J. Combinatorics*, 15:407–410, 1994.
- D. K. Dalai, K. C. Gupta, S. Maitra, “Cryptographically significant Boolean functions: construction and analysis in terms of algebraic immunity,” *FSE 2005*.
- A. Braeken and B. Preneel, “On the algebraic immunity of symmetric Boolean functions,” *INDOCRYPT 2005*.
- E. E. Kummer, “Über die Ergänzungssätze zu den allgemeinen Reciprocitätsgesetzen,” *J. reine angew. Math.*, 44:93–146, 1852.
- F. F. Sellers, M.-Y. Hsiao, L. W. Bearnson, *Error Detecting Logic for Digital Computers*, McGraw-Hill, 1968.